

ADVANCED LINEAR DATA STRUCTURES

List Types, Stacks & Queues

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Course Outline

1 Advanced List Structures

2 Stacks and Queues



Outline

1 Advanced List Structures

2 Stacks and Queues

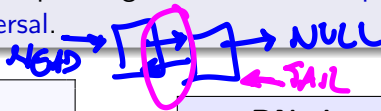


Double-linked List: C Structure



Definition

A **double-linked list** has each node pointing to both **next** and **previous** nodes, enabling **bidirectional traversal**.



```
typedef struct DNode {
    int      data;
    struct DNode *prev;
    struct DNode *next;
} DNode;
```

```
typedef struct {
    DNode *head;
    DNode *tail;
    int    size;
} DList;
```

```
void dl_push_back(DList*, int);
void dl_delete(DList*, DNode*);
DNode *dl_search(DList*, int);
void dl_free(DList*);
```

- data structure

DNode	
data	: int
prev	: DNode*
next	: DNode*

head ↑

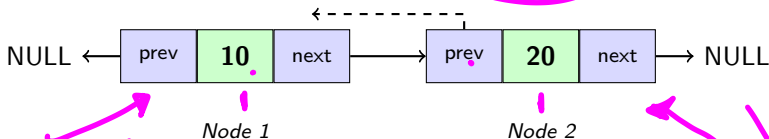
DList	
head	: DNode*
tail	: DNode*
size	: int



Double-linked List: Memory Layout

Node diagram

Each node stores three fields; arrows show **prev/next** links.



Advantages

- Bidirectional traversal
- $O(1)$ delete given a pointer
- Efficient insert at tail

Disadvantages

- Extra memory (pointers/node)
- More pointer updates per op
- Harder to debug

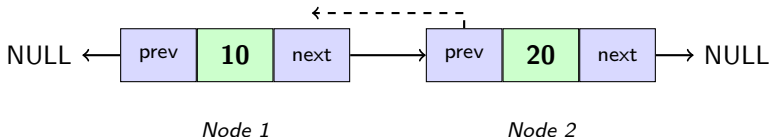
Handwritten notes:
 10 → 20 (with arrows)
 10 ← 20 (with arrows)
 NO PAIRS
 ERROR



Double-linked List: Memory Layout

Node diagram

Each node stores three fields; arrows show **prev**/**next** links.



Advantages

- Bidirectional traversal
- $O(1)$ delete given a pointer
- Efficient insert at tail

Disadvantages

- Extra memory (2 pointers/node)
- More pointer updates per op
- Harder to debug



Double-linked List: C Operations

```

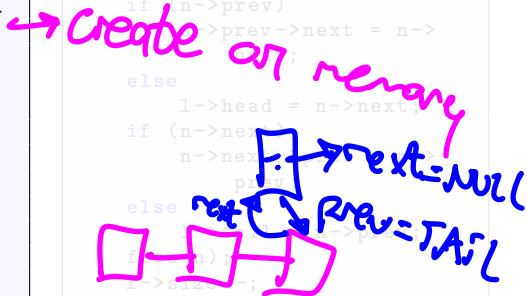
/* Insert at tail */
void dl_push_back(DList *l,
    int v){
    DNode *n = malloc(sizeof(
        DNode));
    n->data = v;
    n->next = NULL;
    n->prev = l->tail;
    if (l->tail)
        l->tail->next = n;
    else
        l->head = n;
    l->tail = n;
    l->size++;
}
~

```

```

/* Delete any node */
void dl_delete(DList *l, DNode
    *n){
    if (n->prev)
        prev->next = n->
    else
        l->head = n->next;
    if (n->next)
        n->next->prev =
    else
        l->tail = n->prev;
    free(n);
    l->size--;
}

```



Key insight

`dl_delete` is $O(1)$: with a pointer to the node, no search is needed.

Double-linked List: C Operations

```

/* Insert at tail */
void dl_push_back(DList *l,
    int v){
    DNode *n = malloc(sizeof(
        DNode));
    n->data = v;
    n->next = NULL;
    n->prev = l->tail;
    if (l->tail)
        l->tail->next = n;
    else
        l->head = n;
    l->tail = n;
    l->size++;
}
~

```

```

/* Delete any node */
void dl_delete(DList *l, DNode
    *n){
    if (n->prev)
        n->prev->next = n->
            next; ✓
    else
        l->head = n->next;
    if (n->next)
        n->next->prev = n->
            prev; ✓
    else
        l->tail = n->prev;
    free(n); ✓
    l->size--;
}

```

Key insight

dl_delete is $O(1)$; with a pointer to the node, no search is needed.



Double-linked List: C Operations

```

/* Insert at tail */
void dl_push_back(DList *l,
    int v){
    DNode *n = malloc(sizeof(
        DNode));
    n->data = v;
    n->next = NULL;
    n->prev = l->tail;
    if (l->tail)
        l->tail->next = n;
    else
        l->head = n;
    l->tail = n;
    l->size++;
}
~

```

```

/* Delete any node */
void dl_delete(DList *l, DNode
    *n){
    if (n->prev)
        n->prev->next = n->
            next;
    else
        l->head = n->next;
    if (n->next)
        n->next->prev = n->
            prev;
    else
        l->tail = n->prev;
    free(n);
    l->size--;
}

```

Key insight

dl_delete is $O(1)$ with a pointer to the node, no search is needed.

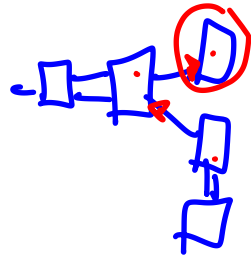
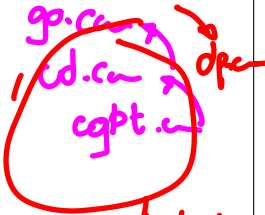
Study Case: Browser Navigation

Problem

Implement **back** / **forward** browser history using a double-linked list.

BrowserHistory

current	: WebPage*
size	: int
+ visit(url : char*)	
+ go_back()	
+ go_forward()	
+ free_history()	



Why Double-linked List?

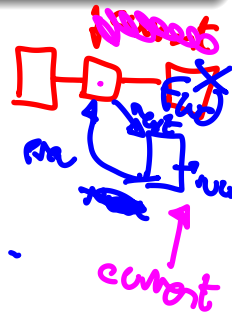
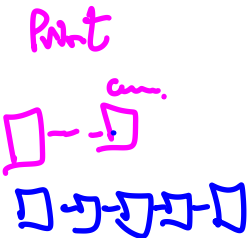
- prev pointer → Back button
- next pointer → Forward button ($O(1)$)
- Visiting a new page: truncate forward history from current

Study Case: Browser Navigation

Problem

Implement **back** / **forward** browser history using a double-linked list.

BrowserHistory	
current	: WebPage*
size	: int
+ visit(url : char*)	
+ go_back()	
+ go_forward()	
+ free_history()	



Why Double-linked List?

- prev pointer → Back button ($O(1)$)
- next pointer → Forward button ($O(1)$)
- Visiting a new page: truncate forward history from current

Circular List: C Structure



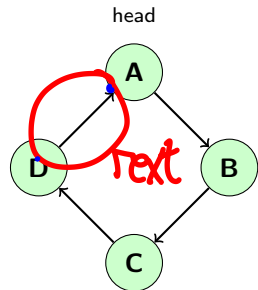
Definition

A **circular list** is a linked list where the **last node's next** points back to the **head**, forming a cycle.

```
typedef struct CNode {
    int data;
    struct CNode *next; /*wraps*/
} CNode;

typedef struct {
    CNode *tail; /*tail->next = head*/
    int size;
} CList;

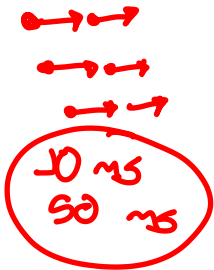
void cl_insert(CList*, int);
void cl_remove(CList*, int);
CNode *cl_next(CList*); /*rotate*/
```



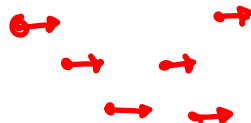
Study Case: Round-Robin Scheduler

Problem

Implement a **round-robin CPU scheduler**: each process receives equal time slices, cycling continuously.



RRScheduler	
current	: Process*
time_quantum	: int
count	: int
+ add_process(pid, burst)	
+ run_quantum()	
+ remove_finished()	
+ print_status()	



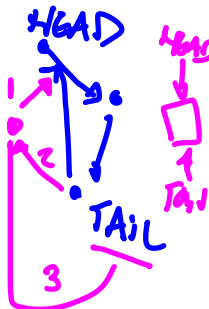
Why Circular List?

The circular structure **naturally rotates**: after the last process, next leads straight back to the first — no restart logic.

Study Case: Round-Robin Scheduler

Problem

Implement a **round-robin CPU scheduler**: each process receives equal time slices, cycling continuously.



RRScheduler	
current	: Process*
time_quantum	: int
count	: int
+ add_process(pid, burst)	
+ run_quantum()	
+ remove_finished()	
+ print_status()	



Why Circular List?

The circular structure **naturally rotates**: after the last process, next leads straight back to the first — no restart logic.

Outline

1 Advanced List Structures

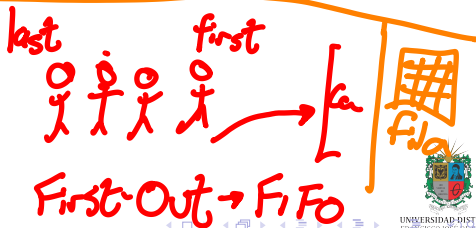
Stacks
(pila)



Last-In First-Out → LIFO

2 Stacks and Queues

Queues
(fila
colas)



First-In First-Out → FIFO



Stack: LIFO Principle and C Structure

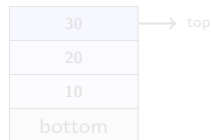
Definition

A **stack** follows **Last-In-First-Out (LIFO)**: both insertions (push) and deletions (pop) occur at the same end, the **top**.

```
typedef struct SNode {
    int data;
    struct SNode *next;
} SNode;

typedef struct {
    SNode *top;
    int size;
} Stack;

void push(Stack *s, int v);
int pop (Stack *s);
int peek(const Stack *s);
int is_empty(const Stack *s);
```



Applications

- Call stack
- Expression evaluation
- Undo systems
- DFS traversal



Stack: LIFO Principle and C Structure

Definition

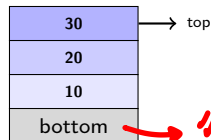
A **stack** follows **Last-In-First-Out (LIFO)**: both insertions (push) and deletions (pop) occur at the same end, the **top**.

```
typedef struct SNode {
    int data;
    struct SNode *next;
} SNode;

typedef struct {
    SNode *top;
    int size;
} Stack;

void push(Stack *s, int v);
int pop(Stack *s);
int peek(const Stack *s);
int is_empty(const Stack *s);
```

Handwritten notes: A red checkmark is next to the SNode struct. A red bracket labeled "Insert" and "Delete" is next to the Stack struct. The word "push" in the function signature is underlined in red.



Handwritten notes: "head" is written in red and underlined. Below it, "insert-ord" and "delete-ord" are written in red. Arrows point from "insert-ord" to "Push" and from "delete-ord" to "Pop".

- Applications:
- Call stack
 - Expression evaluation
 - Undo systems
 - DFS traversal

Stack: LIFO Principle and C Structure

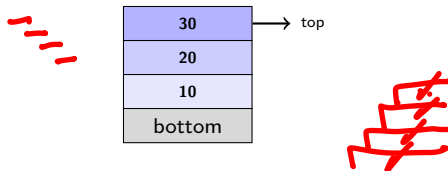
Definition

A **stack** follows **Last-In-First-Out (LIFO)**: both insertions (push) and deletions (pop) occur at the same end, the **top**.

```
typedef struct SNode {
    int data;
    struct SNode *next;
} SNode;

typedef struct {
    SNode *top;
    int size;
} Stack;

void push(Stack *s, int v);
int pop (Stack *s);
int peek(const Stack *s);
int is_empty(const Stack *s);
```



Applications

- Call stack ✓
- Expression evaluation
- Undo systems
- DFS traversal



Stack: C Implementation

```

void push(Stack *s, int v) {
    SNode *n = malloc(sizeof(
        SNode));
    n->data = v;
    n->next = s->top;
    s->top = n;
    s->size++;
}

int is_empty(const Stack *s) {
    return s->top == NULL;
}

void free_stack(Stack *s) {
    while (!is_empty(s))
        pop(s);
}

```

```

int pop(Stack *s) {
    if (!s->top) return -1;
    SNode *tmp = s->top;
    int val = tmp->data;
    s->top = tmp->next;
    free(tmp);
    s->size--;
    return val;
}

int peek(const Stack *s) {
    if (!s->top) return -1;
    return s->top->data;
}

```

Memory safety

Always call `free_stack()` before the stack goes out of scope to avoid memory leaks.

Stack: C Implementation

```

void push(Stack *s, int v) {
    SNode *n = malloc(sizeof(
        SNode));
    n->data = v;
    n->next = s->top;
    s->top = n;
    s->size++;
}

int is_empty(const Stack *s) {
    return s->top == NULL;
}

void free_stack(Stack *s) {
    while (!is_empty(s))
        pop(s);
}

```

```

int pop(Stack *s) {
    if (!s->top) return -1;
    SNode *tmp = s->top;
    int val = tmp->data;
    s->top = tmp->next;
    free(tmp);
    s->size--;
    return val;
}

int peek(const Stack *s) {
    if (!s->top) return -1;
    return s->top->data;
}

```

Memory safety

Always call free_stack() before the stack goes out of scope to avoid memory leaks.

Study Case: Undo / Redo System

Problem

Implement **undo** / **redo** for a text editor using two stacks.

UndoManager
undo_stack : Stack redo_stack : Stack
+ record(action : EditAction) + undo() + redo() + clear()

Logic

- **Record:** push to undo_stack, clear redo_stack
- **Undo:** pop from undo_stack, push to redo_stack
- **Redo:** pop from redo_stack, push to undo_stack

Queue: FIFO Principle and C Structure

Definition

A **queue** follows **First-In-First-Out (FIFO)**: elements are **enqueued** at the rear and **dequeued** from the front.

```
typedef struct QNode {
    int data;
    struct QNode *next;
} QNode;
```

```
typedef struct {
    QNode *front;
    QNode *rear;
    int size;
} Queue;
```

```
void enqueue(Queue *q, int v);
int dequeue(Queue *q);
int q_front(const Queue *q);
int q_empty(const Queue *q);
```



Applications

- CPU scheduling
- I/O buffers
- BFS traversal
- Service queues

Queue: FIFO Principle and C Structure

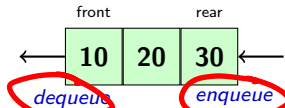
Definition

A **queue** follows **First-In-First-Out (FIFO)**: elements are **enqueued** at the **rear** and **dequeued** from the **front**.

```
typedef struct QNode {
    int data;
    struct QNode *next;
} QNode;

typedef struct {
    QNode *front;
    QNode *rear;
    int size;
} Queue;

void enqueue(Queue *q, int v);
int dequeue(Queue *q);
int q_front(const Queue *q);
int q_empty(const Queue *q);
```



insert end → delete begin
insert begin → delete end

- CPU scheduling
- FIFO buffers
- BFS traversal
- Service queues

Queue: FIFO Principle and C Structure

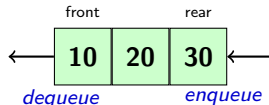
Definition

A **queue** follows **First-In-First-Out (FIFO)**: elements are **enqueued** at the **rear** and **dequeued** from the **front**.

```
typedef struct QNode {
    int data;
    struct QNode *next;
} QNode;

typedef struct {
    QNode *front;
    QNode *rear;
    int size;
} Queue;

void enqueue(Queue *q, int v);
int dequeue(Queue *q);
int q_front(const Queue *q);
int q_empty(const Queue *q);
```



Applications

- CPU scheduling
- I/O buffers
- BFS traversal
- Service queues

Round Robin

Queue: C Implementation

```

void enqueue(Queue *q, int v)
{
    QNode *n = malloc(sizeof(
        QNode));
    n->data = v;
    n->next = NULL;
    if (q->rear)
        q->rear->next = n;
    else
        q->front = n;
    q->rear = n;
    q->size++;
}

```

```

int dequeue(Queue *q) {
    if (!q->front)
        return -1;
    QNode *tmp = q->front;
    int val = tmp->data;
    q->front = tmp->next;
    if (!q->front)
        q->rear = NULL;
    free(tmp);
    q->size--;
    return val;
}
~

```

Key difference from Stack

Stack maintains one pointer (top). Queue maintains two: front and rear.



Queue: C Implementation

```
void enqueue(Queue *q, int v)
{
    QNode *n = malloc(sizeof(
        QNode));
    n->data = v;
    n->next = NULL;
    if (q->rear)
        q->rear->next = n;
    else
        q->front = n;
    q->rear = n;
    q->size++;
}
```

```
int dequeue(Queue *q) {
    if (!q->front)
        return -1;
    QNode *tmp = q->front;
    int val = tmp->data;
    q->front = tmp->next;
    if (!q->front)
        q->rear = NULL;
    free(tmp);
    q->size--;
    return val;
}
~
```

Key difference from Stack

Stack maintains one pointer (top). Queue maintains two: front and rear.



Study Case: Bank Queue System

Problem

Implement a **bank teller queue**: customers arrive and are served **in order of arrival**.

BankQueue	
queue	: Queue
next_ticket	: int
+ take_ticket(name : char*)	
+ call_next()	
+ get_wait_count()	
+ print_status()	

Why Queue?

FIFO guarantees **fair service**: the first customer to arrive is the first to be served. No customer can be skipped.

Study Case: Bank Queue System

Problem

Implement a **bank teller queue**: customers arrive and are served **in order of arrival**.

BankQueue	
queue	: Queue
next_ticket	: int
+ take_ticket(name : char*)	
+ call_next()	
+ get_wait_count()	
+ print_status()	

Why Queue?

FIFO guarantees **fair service**: the first customer to arrive is the first to be served. No customer can be skipped.

Outline

1 Advanced List Structures

2 Stacks and Queues



Complexity Summary

Operation	Time	Notes
Stack push	$O(1)$	Prepend to top
Stack pop	$O(1)$	Remove from top
Stack peek	$O(1)$	Read without removal
Queue enqueue	$O(1)$	Append to rear
Queue dequeue	$O(1)$	Remove from front
DList insert at ends	$O(1)$	Requires tail pointer
DList dl_delete	$O(1)$	Pointer to node given
DList dl_search	$O(n)$	Sequential scan

Space complexity

All linked implementations: $O(n)$ for n elements.

Overhead: 1 pointer / node (singly), 2 pointers / node (doubly).



Complexity Summary

Operation	Time	Notes
Stack push	$O(1)$	Prepend to top
Stack pop	$O(1)$	Remove from top
Stack peek	$O(1)$	Read without removal
Queue enqueue	$O(1)$	Append to rear
Queue dequeue	$O(1)$	Remove from front
DList insert at ends	$O(1)$	Requires tail pointer
DList dl_delete	$O(1)$	Pointer to node given
DList dl_search	$O(n)$	Sequential scan

Space complexity

All linked implementations: $O(n)$ for n elements.

Overhead: 1 pointer / node (singly), 2 pointers / node (doubly).



Outline

1 Advanced List Structures

2 Stacks and Queues



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-i>

